Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 16]
[Atualizado em: 14/08/2025]

[VITE – Criando Funções]

1

CRIANDO FUNÇÕES

NOMENCLATURAS – CONVENÇÕES

- ✓ **Variáveis**
  - ✓ **Formato**: camelCase
  - ✓ **Exemplo**: userName, isLoggedIn, totalPrice
  - ✓ **Regras**:
    - ✓ Use nomes descritivos (evite x, data, temp, nomes sem contexto).
    - ✓ Prefira nomes curtos, mas claros.
    - ✓ Use prefixos como is, has, can para variáveis booleanas: isActive, hasError.

NOMENCLATURAS – CONVENÇÕES

- ✓ **Funções**
  - ✓ **Formato**: camelCase
  - ✓ **Exemplo**: handleClick, fetchUserData, calculateTotal
  - ✓ **Regras**:
    - ✓ Comece com um verbo que indique ação (get, fetch, handle, submit, update, etc).
    - ✓ Seja específico: handleLogin, não apenas handle.

NOMENCLATURAS – CONVENÇÕES

- ✓ **Componentes React (incluindo páginas em Vite.js)**
  - ✓ **Formato**: PascalCase
  - ✓ **Exemplo**: Header, UserProfile, RecipeCard
  - ✓ **Regras**:
    - ✓ Cada componente deve estar em seu próprio arquivo com o mesmo nome: RecipeCard.tsx → function RecipeCard() { ... }
    - ✓ Evite abreviações sem contexto (ex: UsrPrf).
    - ✓ Componentes com múltiplas palavras devem manter a clareza: ProductListItem, UserSettingsForm.

NOMENCLATURAS – CONVENÇÕES

- ✓ **Pastas e arquivos**
  - ✓ **Pastas**
    - ✓ **Formato**: Geralmente em kebab-case ou camelCase (depende da equipe/projeto, mas seja consistente).
    - ✓ **Exemplo**: components/, user-profile/, recipePage/
  - ✓ **Arquivos**
    - ✓ **Componentes React**: PascalCase.tsx (como LoginForm.tsx)
    - ✓ **Hooks**: useCamelCase.ts (como useAuth.ts)
    - ✓ **Utilitários**: camelCase.ts (como formatDate.ts)
    - ✓ **Páginas** (em /pages): o nome do arquivo define a rota (ex: about.tsx → /about

NOMENCLATURAS – CONVENÇÕES

- ✓ **Hooks Personalizados**
  - ✓ **Formato**: useCamelCase
  - ✓ **Exemplo**: useAuth, useFetch, useToggle
  - ✓ **Regras**:
    - ✓ Sempre comece com *use*.
    - ✓ Mantenha o nome descritivo do propósito do *hook*.

NOMENCLATURAS – CONVENÇÕES

- ✓ **Constantes**
  - ✓ **Formato**: UPPER_SNAKE_CASE (para valores fixos ou de configuração)
  - ✓ **Exemplo**: API_BASE_URL, MAX_RETRIES, DEFAULT_LANGUAGE

NOMENCLATURAS – CONVENÇÕES

- ✓ **Tipos e Interfaces**
  - ✓ **Formato**: PascalCase
  - ✓ **Exemplo**: User, ProductProps, AuthContextType

NOMENCLATURAS – CONVENÇÕES

✓ Resumo

Tipo	Convenção	Exemplo
Variáveis	camelCase	userName
Funções	camelCase	fetchUserData()
Componentes	PascalCase	UserProfile
Pasta/Arquivos	kebab-case ou camelCase	user-profile/
Hooks Personalizados	useCamelCase	useTheme
Constantes	UPPER_SNAKE_CASE	API_URL
Tipos e Interfaces	PascalCase	UserProps

2

VITE – FUNÇÕES

FUNÇÃO NOMEADA COM EXPORTAÇÃO DIRETA

- ✓ Em projetos React, é possível declarar componentes funcionais de diferentes maneiras.
- ✓ Todas essas formas retornam elementos JSX e são perfeitamente compatíveis com o Vite, que atua como uma ferramenta de desenvolvimento e ***build*** — ou seja, ele não impõe um estilo específico de função.
- ✓ Apesar de servirem ao mesmo propósito — construir partes reutilizáveis da interface — essas funções podem ser escritas com sintaxes distintas, cada uma com suas próprias vantagens, desvantagens e melhores contextos de uso.
- ✓ Entender essas variações ajuda a escrever código mais claro, organizado e adequado ao estilo do projeto ou equipe.

FUNÇÃO NOMEADA COM EXPORTAÇÃO DIRETA

- ✓ Esta abordagem utiliza uma declaração de função nomeada (Function Declaration) combinada com uma exportação padrão (export default). Isso significa que a função é definida no escopo global do módulo e pode ser exportada diretamente como o valor padrão do arquivo.

- ✓ **Características Técnicas:**
  - ✓ **Declaração Nomeada**: A função é definida com um nome explícito (NomeFuncao), o que facilita debugging e rastreamento no VITE DevTools.
  - ✓ **Escopo do Módulo**: A função existe apenas dentro do módulo onde foi declarada.
  - ✓ **Exportação Padrão**: Permite importar o componente sem a necessidade de chaves {}, o que simplifica a sintaxe de importação.
  - ✓ **Hoisting**: Como é uma Function Declaration, ela sofre hoisting, ou seja, pode ser chamada antes da sua definição no código.

FUNÇÃO NOMEADA COM EXPORTAÇÃO DIRETA

- Sintaxe:

```
export default function NomeFuncao() {
    return (
        <>
            <div>Olá, mundo!</div>;
        </>
    )
}
```

FUNÇÃO NOMEADA COM EXPORTAÇÃO DIRETA

- **Vantagens:**
  - Código mais limpo e direto para páginas VITE.
  - Permite importar sem precisar de chaves { }.
  - Melhor compatibilidade com a convenção do VITE para páginas.

- **Desvantagens:**
  - Só pode haver um export default por arquivo.
  - O nome da função pode ser alterado ao importar, dificultando rastreamento no código.
    - `import MinhaPagina from "./pagina"; // Nome diferente do original`

FUNÇÃO NOMEADA SEM EXPORTAÇÃO DIRETA

- ✓ Uma Named Function Declaration é uma função declarada explicitamente com um identificador, permitindo sua reutilização no escopo do módulo. Quando a exportação não é feita diretamente na declaração, a função pode ser referenciada e manipulada antes de ser exportada.

- ✓ **Características Técnicas:**
  - ✓ Definida usando function NomeDaFuncao() { }.
  - ✓ A função é içada (hoisted), permitindo sua chamada antes da definição no código.
  - ✓ A exportação ocorre separadamente via export default NomeDaFuncao ou export { NomeDaFuncao }.
  - ✓ Mantém um nome identificável no DevTools do VITE, facilitando debugging.
  - ✓ Permite exportar múltiplas funções do mesmo arquivo quando usada com export {}.

FUNÇÃO NOMEADA SEM EXPORTAÇÃO DIRETA

- ✓ Sintaxe:

```
function NomeFuncao() {
    return (
        <>
            <div>Olá, mundo!</div>;
        </>
    )
}
export default NomeFuncao;
```

FUNÇÃO NOMEADA SEM EXPORTAÇÃO DIRETA

- ✓ **Vantagens:**
  - ✓ Permite declarar a função primeiro, facilitando debugging e testes.
  - ✓ Fácil de ler e entender.
  - ✓ Nome da função aparece corretamente no DevTools.

- ✓ **Desvantagens:**
  - ✓ Ainda tem a limitação de apenas um export default por arquivo.
  - ✓ Pode ser um pouco mais verboso.

FUNÇÃO ARROW FUNCTION

- ✓ Uma Arrow Function é uma expressão de função introduzida no ES6 que utiliza a sintaxe => para definir funções de forma mais concisa.

  - ✓ **Características Técnicas:**
    - ✓ Sintaxe reduzida e mais legível em comparação com function.
    - ✓ Léxico de this: não cria seu próprio contexto de this, herdando do escopo onde foi definida.
    - ✓ Não pode ser usada como construtora (new não é permitido).
    - ✓ Não possui arguments, super ou new.target.
    - ✓ Pode ser escrita de forma implícita (retorno automático) ou explícita (usando {} e return).

FUNÇÃO ARROW FUNCTION

- Sintaxe:

```
const NomeFuncao= () => {
    return (
        <>
            <div>Olá, mundo!</div>;
        </>
    )
}
export default NomeFuncao;
```

FUNÇÃO ARROW FUNCTION

- ✓ **Vantagens:**
  - ✓ Sintaxe moderna e mais curta, ideal para componentes funcionais simples.
  - ✓ Pode ser usada com VITE Hooks sem problemas.
  - ✓ Elimina a necessidade de bind(this), útil em classes (não aplicável a funções puras).

- ✓ **Desvantagens:**
  - ✓ Pode dificultar debugging porque o nome da função pode ser undefined no DevTools do VITE.
  - ✓ Não permite reatribuição, pois é uma constante (const).

FUNÇÃO NOMEADA SEM EXPORTAÇÃO PADRÃO

- ✓ Uma Named Function Declaration é uma função declarada explicitamente com um identificador, permitindo sua reutilização dentro do módulo. Quando exportada sem ser padrão (default), ela deve ser importada pelo mesmo nome dentro de chaves {}.

- ✓ **Características Técnicas:**
  - ✓ Declarada com function NomeDaFuncao() {} e exportada via export { NomeDaFuncao }.
  - ✓ Suporta a exportação de múltiplas funções do mesmo módulo.
  - ✓ Mantém o nome original durante a importação, garantindo rastreabilidade no código.
  - ✓ Sujeita a hoisting, permitindo que seja chamada antes da definição.
  - ✓ Melhora a identificação no DevTools do VITE, útil para debugging.
  - ✓ Não permite alteração do nome no momento da importação, garantindo consistência entre módulos.

FUNÇÃO NOMEADA SEM EXPORTAÇÃO PADRÃO

- ✓ Sintaxe:

```
export function NomeFuncao() {
    return (
        <>
            <div>Olá, mundo!</div>;
        </>
    )
}
```

FUNÇÃO NOMEADA SEM EXPORTAÇÃO PADRÃO

- **Vantagens:**
  - Permite exportar várias funções do mesmo arquivo:
    - export function ComponenteA() { }
    - export function ComponenteB() { }
  - Melhora a organização e facilita testes.
  - Mantém o nome correto da função no DevTools.
  - Permite exportar múltiplas funções ou utilitários do mesmo arquivo.
  - Útil para bibliotecas ou conjuntos de componentes.

- **Desvantagens:**
  - Importação é mais verbosa, exigindo { }
    - import { NomeFuncao } from "./caminho";
  - Se o nome mudar na exportação, a importação também precisa ser alterada.

3

VITE – CONCLUSÃO

CONCLUSÃO

- ✓ No **Vite**, as funções desempenham papéis fundamentais na construção da interface e na lógica da aplicação. Existem diversas formas de definir funções, cada uma com características técnicas distintas, influenciando o comportamento do código em termos de modularidade, reutilização, otimização e depuração.
  - ✓ **Funções Nomeadas** são ideais para rastreabilidade e debugging, pois mantêm seus nomes no DevTools do React.
  - ✓ **Arrow Functions** oferecem sintaxe concisa e escopo léxico de this, sendo amplamente usadas em componentes funcionais e callbacks.
  - ✓ **Exportações Padrão** (export default) simplificam a importação, mas permitem renomeação, o que pode impactar a rastreabilidade.
  - ✓ **Exportações Nomeadas** (export {}) garantem maior consistência entre módulos, facilitando a manutenção e organização do código.
- ✓ A escolha entre essas abordagens deve considerar fatores como clareza do código, necessidade de reutilização e padrões de arquitetura adotados no projeto.

CONCLUSÃO

Comparativo entre Formas de Criar Componentes

Forma de Declaração	Exemplo	Vantagens	Desvantagens	Quando Usar

Função Nomeada COM Exportação Direta	**export default function NomeFuncao()** {
	return (
	
	 Olá, mundo! ;
	</>
	)
	}
		• Nome visível no DevTools Boa legibilidade
		• Familiar para iniciantes
			• Mais verbosa que arrow function	• Ideal para ensino e projetos simples

Função nomeada SEM Exportação Direta	**function NomeFuncao() {** return (Olá, mundo! ; </>) } **export default NomeFuncao;**	• Útil em arquivos com várias funções • Clareza na definição	• Inutilizável fora do arquivo se não for exportada	• Quando será exportada depois com export {}
Arrow Function	**const NomeFuncao= () => {** return (Olá, mundo! ; </>) } **export default NomeFuncao**;	• Moderna • Sintaxe curta • Boa com hooks	• Nome não aparece facilmente no DevTools	• Muito usada em projetos atuais e com hooks

Função Nomeada SEM Exportação Padrão	**export function NomeFuncao()** {
	return (
	
	 Olá, mundo! ;
	</>
	)
	}
		• Flexível para múltiplas exports
		• Organiza melhor arquivos grandes
			• Importação precisa de chaves: import { MeuComponente }	• Ideal em bibliotecas ou arquivos com várias funções

4

VITE – FUNÇÕES – EXERCÍCIOS

VITE – EXERCÍCIOS

- ✓ **Exercício 01 - Exploração das 4 Formas de Definir Funções no VITE**
  - ✓ O objetivo deste exercício é praticar e entender as 4 formas principais de definir funções em **VITE** e **React**, criando componentes.
  - ✓ Criação de Componentes:
    - ✓ Crie um componente com função nomeada e exportação padrão.
      - ✓ Crie um componente chamado Saudacao que exibe "Olá, Mundo!" na tela.
      - ✓ Exporte esse componente como default.
    - ✓ Crie um componente com função nomeada e exportação nomeada.
      - ✓ Crie um componente chamado Mensagem que exibe "Bem-vindo ao VITE!".
      - ✓ Exporte esse componente com a exportação nomeada.
    - ✓ Crie um componente usando uma arrow function com exportação padrão.
      - ✓ Crie um componente chamado BoasVindas usando uma arrow function, que exibe "Boas-vindas ao React!".
      - ✓ Exporte esse componente como default.
    - ✓ Crie um componente usando função nomeada sem exportação padrão.
      - ✓ Crie um componente chamado Info que exibe "Isso é simples!".
      - ✓ Exporte esse componente de forma nomeada (sem usar export default).

Referências Bibliográficas

ABREU, Luis. Typescript - O Javascript moderno para criação de aplicações. Editora FCA, 2017.

ADRIANO, T. S. Guia prático de TypeScript. São Paulo: Casa do Código, 2021.

ANTONIO, C. Pro React: Build Complex Front-End Applications in a Composable Way With React. Apress, 2015.

BABEL. Babel Handbook. Babel, 2025. Disponível em: https://babeljs.io/docs/en/. Acesso em: 14 de julho de 2025.

BANKS, Alex; PORCELLI, Eve. React: Design de Interfaces com JavaScript. São Paulo: Novatec, 2020.

BOSWELL, D; FOUCHER, T. The Art of Readable Code: Simple and Practical Techniques for Writing Better Code. Estados Unidos: O'Reilly Media, 2012.

BRITO, Robin Cris. Android Com Android Studio - Passo A Passo. Editora Ciência Moderna.

BUNA, S. React Succinctly. Estados Unidos: [s.n], 2016. Disponível em: <www.syncfusion.com/ebooks/reactjs_succinctly>. Acesso em: 07 de fevereiro de 2025.

CHIN, John. VITE Quick Start Guide: Server-side rendering done right. Birmingham: Packt Publishing, 2019.

DEV COMMUNITY. Understanding Arrow Functions in JavaScript. DEV.to, 2023. Disponível em: https://dev.to/somedood/understanding-arrow-functions-in-javascript-3mjg. Acesso em: 14 de julho de 2025.

DUCKETT, Jon. JavaScript e jQuery: desenvolvimento de interfaces web interativas. Rio de Janeiro: Alta Books, 2015.

FACEBOOK (2019a). React: Getting Started. React Docs, 2019. Disponível em: <reactjs.org/docs/react-api.html>. Acesso em: 07 de fevereiro de 2025.

FACEBOOK (2019b). React Without ES6. React Docs, 2019. Disponível em: <reactjs.org/docs/react-without-es6.html>. Acesso em: 07 de fevereiro de 2025.

FACEBOOK (2019c). React Without JSX. React Docs, 2019. Disponível em: <reactjs.org/docs/react-without-jsx.html>. Acesso em: 07 de fevereiro de 2025.

FLANAGAN, David. JavaScript: o guia definitivo. 7. ed. São Paulo: Novatec, 2021.

FREECODECAMP. JavaScript Function Declaration vs. Expression. FreeCodeCamp, 2023. Disponível em: https://www.freecodecamp.org/news/javascript-function-declaration-vs-function-expression/. Acesso em: 14 de julho de 2025.

FREEMAN, Eric ROBSON, Elisabeth. Use a Cabeça! Programação em HTML5. Rio de Janeiro: Editora Alta Books, 2014

FREEMAN, Eric; ROBSON, Elisabeth. Use a Cabeça! HTML e CSS. 2. ed. São Paulo: Alta Books, 2015.

Referências Bibliográficas

GACKENHEIMER, C. Introduction to React: Using React to Build scalable and efficient user interfaces.[s.i.]: Apress, 2015.

GEEKSFORGEEKS. Different ways to declare functions in JavaScript. GeeksforGeeks, 2024. Disponível em: https://www.geeksforgeeks.org/different-ways-to-declare-functions-in-javascript/. Acesso em: 14 de julho de 2025.

GOLDBERG, Josh. Aprendendo TypeScript: Melhore Suas Habilidades de Desenvolvimento web Usando Criando funções Type-Safe. São Paulo: Novatec. 2022

HUDSON, P. Hacking with React. 2016. Disponível em: <www.hackingwithreact.com/read/1/3/introduction-to-jsx>. Acesso em: 07 de fevereiro de 2025.

JOHNSON, Adam. Modern Frontend Development with React. Birmingham: Packt Publishing, 2021.

KOSTRZEWA, D. Is React.js the Best Criando funções Framework in 2018? 2018. Disponível em: <hackernoon.com/is-react-js-the-best-JavaScript-framework-in-2018-264a0eb373c8>. Acesso em: 07 de fevereiro de 2025.

LERNA, Allen. Programming TypeScript. Sebastopol: O'Reilly Media, 2020.

MARTIN, R. Clean Code: A Handbook of Agile Software Craftsmanship. Estados Unidos: Prentice Hall, 2009.

MDN Web Docs. JavaScript Functions. Mozilla, 2024. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Functions. Acesso em: 14 de julho de 2025.

MURRAY, Raymie. TypeScript Quickly. Shelter Island: Manning Publications, 2020.

ReactJS. React Documentation – Learn React. Meta, 2025. Disponível em: https://react.dev/learn. Acesso em: 14 de julho de 2025.

REIS, Loiane Groner. JavaScript moderno: domine a linguagem do navegador. São Paulo: Novatec, 2020.

SWC PROJECT. Speedy Web Compiler. SWC, 2025. Disponível em: https://swc.rs. Acesso em: 14 de julho de 2025.

TYPESCRIPT. Documentation – TypeScript for JavaScript Programmers. Microsoft, 2025. Disponível em: https://www.typescriptlang.org/docs/. Acesso em: 14 de julho de 2025.

VITEJS. Guide – Getting Started. Vite, 2025. Disponível em: https://vitejs.dev/guide/. Acesso em: 14 de julho de 2025.

W3SCHOOLS. JavaScript Function Syntax. W3Schools, 2025. Disponível em: https://www.w3schools.com/js/js_function_definition.asp. Acesso em: 14 de julho de 2025.

WIERUCH, Robin. The Road to React: Your Journey to Master React.js in JavaScript. 2021.

ZAKAS, Nicholas C. Understanding ECMAScript 6. San Francisco: No Starch Press, 2016.

DÚVIDAS?

CRÍTICAS?

SUGESTÕES?

AMEAÇAS?